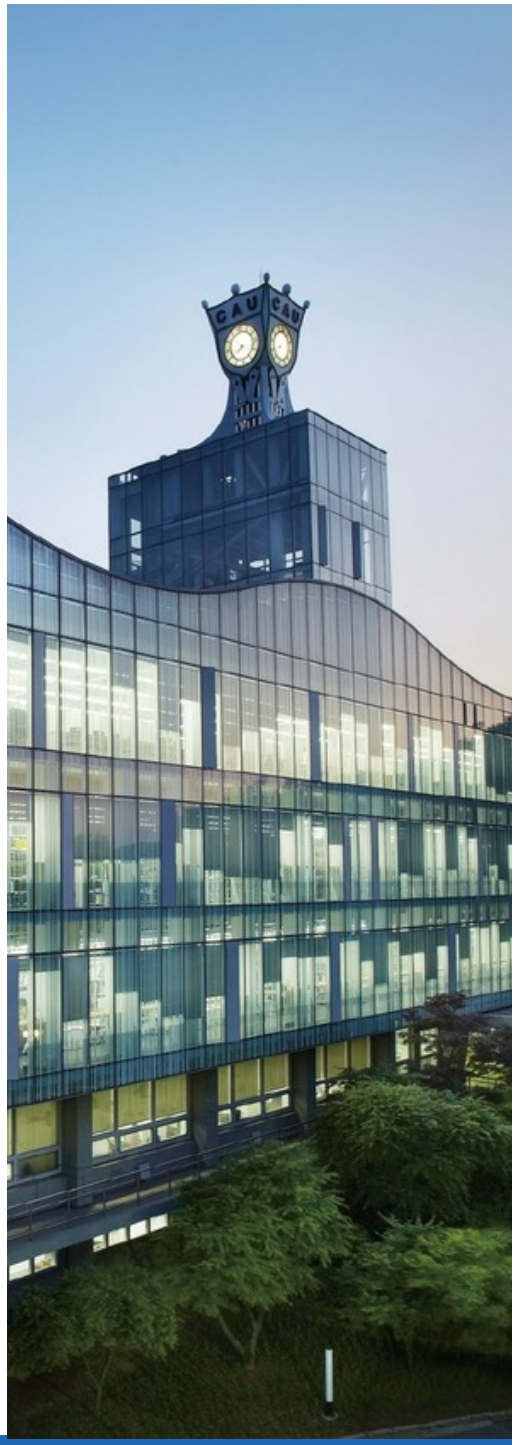
Principles of Programming Languages

Lecture 3
Describing Syntax and Semantics (구문과 의미론)

Jinsung Kim, Ph.D.
High-Performance Computing (HPC) Lab
Computer Science and Engineering
Chung-Ang University



Topics

- Introduction
- The General Problem of Describing Syntax
- Formal Methods of Describing Syntax
- Attribute Grammars
- Describing the Meanings of Programs: Dynamic Semantics
- Summary

Introduction

- **Syntax (구문)**
  - The form or structure of the expressions (표현식), statements (문장), and program units (프로그램 단위)
  - The syntax of a Java, while statement:

    ```
    while (bool_expr) statement
    ```

- **Semantics (의미론)**
  - The meaning of the expressions, statements, and program units
  - When the current value of the Boolean expression (`bool_expr`) is true, the embedded statement is executed
  - If the Boolean expression (`bool_expr`) is false, control transfers to the statement following the while construct

Introduction

- **Syntax and semantics provide a language's definition**
  - Users of a language definition
    - Other language designers
      - To identify ambiguities, inconsistencies, and design flaws.
    - Implementers
      - To implement language construct correctly.
    - Programmers (the users of the language)
      - To use statements and constructs correctly and effectively.

The General Problem of Describing Syntax: Terminology

- A *sentence* (문장) is a string of characters over some alphabet (Σ = {allowed symbols})
- A *language* (언어) is a set of sentences
- A *lexeme* (어휘항목) is the lowest level syntactic unit of a language
  - e.g., `*`, `sum`, `begin`
- A *token* (토큰) is a category of lexemes
  - e.g., `identifier`

The General Problem of Describing Syntax: Terminology

- **Example**

```
index = 2 * count + 17;
```

Lexemes	*Tokens*
`index`	identifier
`=`	equal_sign
`2`	int_literal
`*`	mult_op
`count`	identifier
`+`	plus_op
`17`	int_literal
`;`	semicolon

Formal Definition of Languages

- **Recognizers (언어 인식기) // 기계, 시스템**
  - A recognition device (R) reads input strings over the alphabet (Σ) of the language (L) and decides whether the input strings belong to the language
  - Example: syntax analysis part of a compiler
    - Detailed discussion of syntax analysis appears in Ch.4

- **Generators (언어 생성기) // 사람**
  - A device that generates sentences of a language
  - One can determine if the syntax of a particular sentence is syntactically correct by comparing it to the structure of the generator

Formal Methods of Describing Syntax

- **BNF and CFGs (Context-Free Grammars)**
  - Context-Free Grammars (문맥-자유 문법)
    - Developed by Noam Chomsky in the mid-1950s
    - Language generators, meant to describe the syntax of natural languages
    - Define a class of languages called context-free languages
  - Backus-Naus Form (1959)
    - Invented by John Backus to describe the syntax of Algol 58
    - A formal notation for specifying programming language syntax
    - BNF is equivalent to context-free grammars

BNF Fundamentals (1/2)

- **Metalanguage (메타 언어)**
- **In BNF, abstractions are used to represent classes of syntactic structures---they act like syntactic variables (also called nonterminal symbols, or just terminals)**

```
<assign> → <var> = <expression> // rule or production
```

- **A rule has a left-hand side (LHS), which is a nonterminal, and a right-hand side (RHS), which is a string of terminals and/or nonterminals**
- **Terminals are lexemes or tokens**

BNF Fundamentals (2/2)

- **Nonterminals are often enclosed in angle brackets**
  - Examples of BNF rules

```
<ident_list> → identifier
             | identifier, <ident_list>
<assign>     → <var> = <expr>
```

- **A BNF description (or Grammar) is a finite non-empty set of rules**

- **A start symbol**
  - A special nonterminal of a grammar
  - The sentences of a language are generated through a sequence of applications of the rules, beginning with a start symbol

BNF Rules

- **An abstraction (or nonterminal symbol) can have more than one RHS**

```
<if_stmt> → if (<logical_expr>) <stmt>
<if_stmt> → if (<logical_expr>) <stmt> else <stmt>

<if_stmt> → if (<logical_expr>) <stmt>
          | if (<logical_expr>) <stmt> else <stmt>
```

Describing Lists

- **Variable-length syntactic lists are described using *recursion***
  - A rule is recursive if its LHS appears in its RHS
  - Example of a list of identifier:

```
<ident_list> → ident
             | ident, <ident_list>
```

- **A derivation is a repeated application of rules**
  - Start with the start symbol
  - End with a sentence consisting of terminal symbols only

An Example Grammar

- **Example**

```
<program> → begin <stmts> end
<stmts>   → <stmt>
          | <stmt> ; <stmts>
<stmt>    → <var> = <expr>
<var>     → a|b|c|d
<expr>    → <var> + <var>
          | <var> - <var>
          | <var>
```

An Example Derivation

- **Example: begin** a = b + c; b = c **end**

<program> ⇒ **begin** <stmts> **end**
⇒ **begin** <stmt>; <stmts> **end**
⇒ **begin** <var> = <expr>; <stmts> **end**
⇒ **begin** a = <expr> ; <stmts> **end**
⇒ **begin** a = <var> + <var> ; <stmts> **end**
⇒ **begin** a = b + <var> ; <stmts> **end**
⇒ **begin** a = b + c ; <stmts> **end**
⇒ **begin** a = b + c ; <stmt> **end**
⇒ **begin** a = b + c ; <var> = <expr> **end**
⇒ **begin** a = b + c ; b = <expr> **end**
⇒ **begin** a = b + c ; b = <var> **end**
⇒ **begin** a = b + c ; b = c **end**

Derivations

- **Every string of symbols in a derivation is called a *sentential form* (문장 형태)**
- **A *sentence* (문장) is a sentential form that has *only* terminal symbols**
- **A leftmost derivation (최좌단 유도) is one in which the leftmost nonterminal in each sentential form is to be expanded**
- **A derivation may be neither leftmost nor rightmost**
  - Derivation order has **no** effect on the language generated by a grammar

Parse Tree

- **Parse Tree**
  - A tree representation that shows how a sentence is derived from the start symbol of a grammar by applying its production rules
- **Example:**

```
<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <id> = <expr>
        | <id> * <expr>
        | ( <expr> )
        | <id>
```

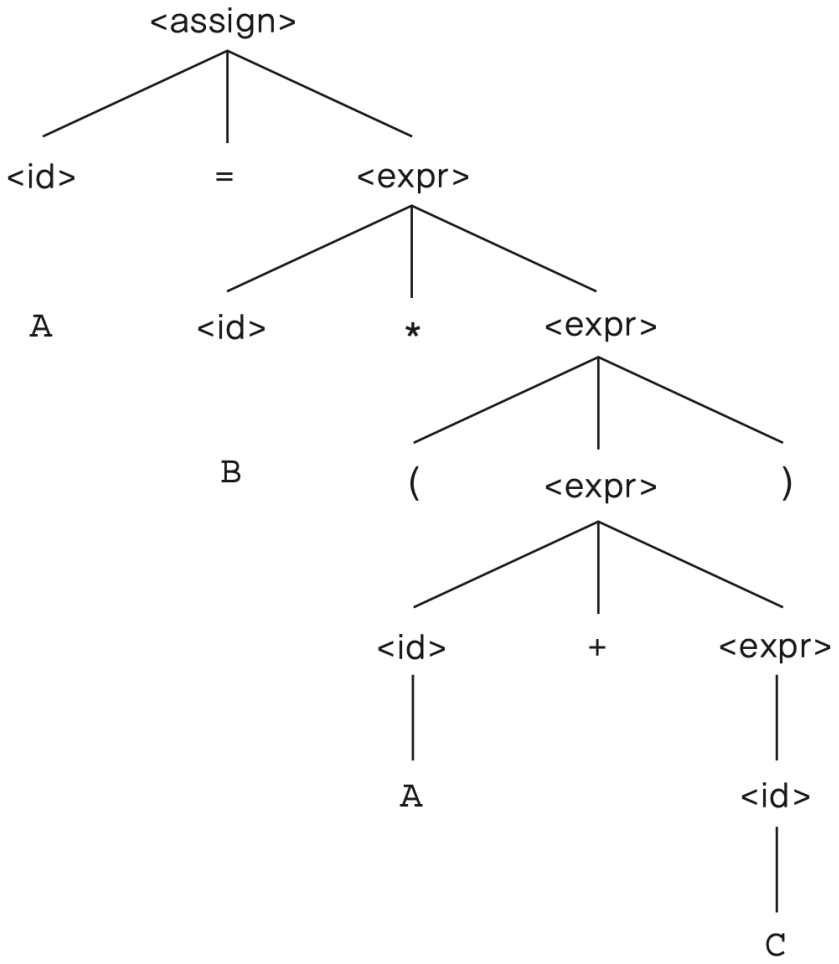
Parse Tree

- **Example:** `A = B * (A + C)`

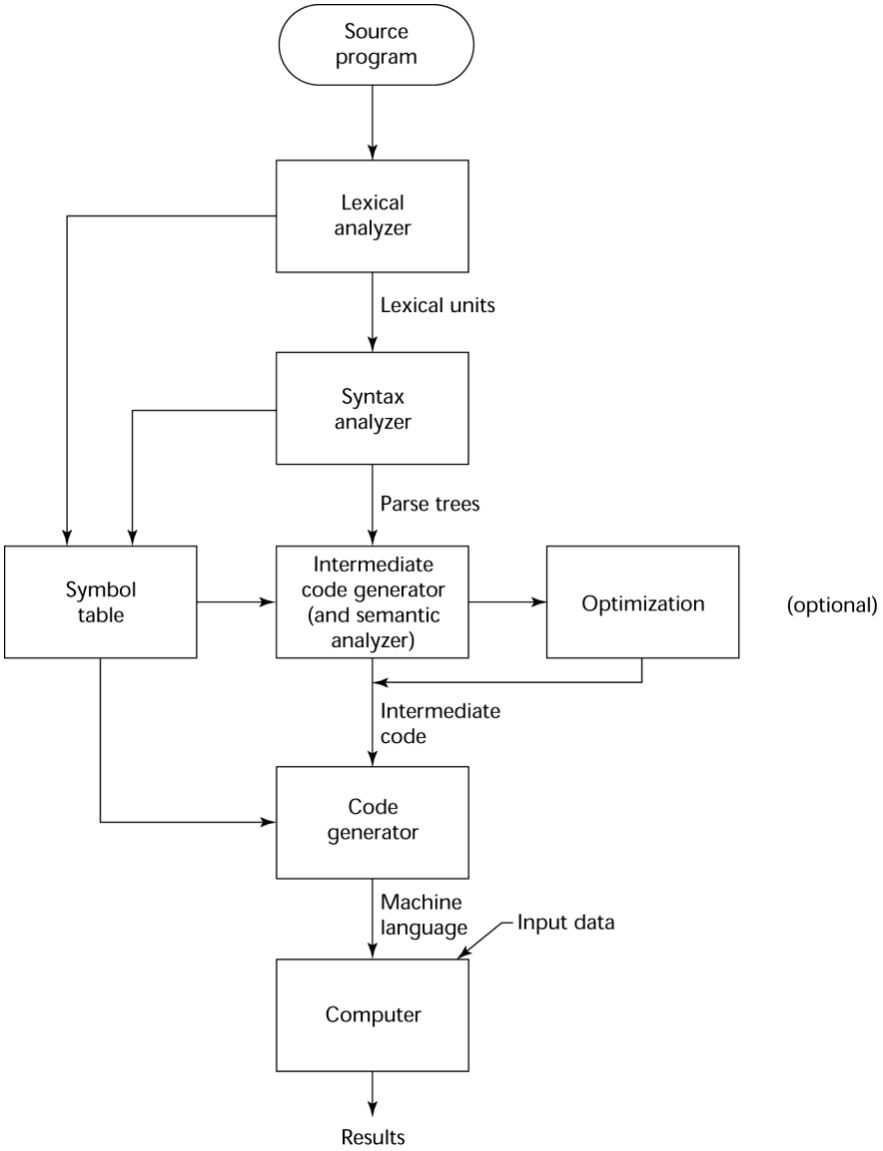
```
<assign>  ⇒ <id> = <expr>
          ⇒ A = <expr>
          ⇒ A = <id> * <expr>
          ⇒ A = B * <expr>
          ⇒ A = B * ( <expr> )
          ⇒ A = B * ( <id> + <expr> )
          ⇒ A = B * ( A + <expr> )
          ⇒ A = B * ( A + <id> )
          ⇒ A = B * ( A + C )
```

Parse Tree

- **Example:** `A = B * (A + C)`



Compilation Process (Recap)



Ambiguity(모호성) in Grammars

- **A grammar is *ambiguous* (모호하다) if and only if it generates a sentential form that has *two or more* distinct parse trees**
  - If the grammar generates a sentence with more than one leftmost derivation
  - If the grammar generates a sentence with more than one rightmost derivation

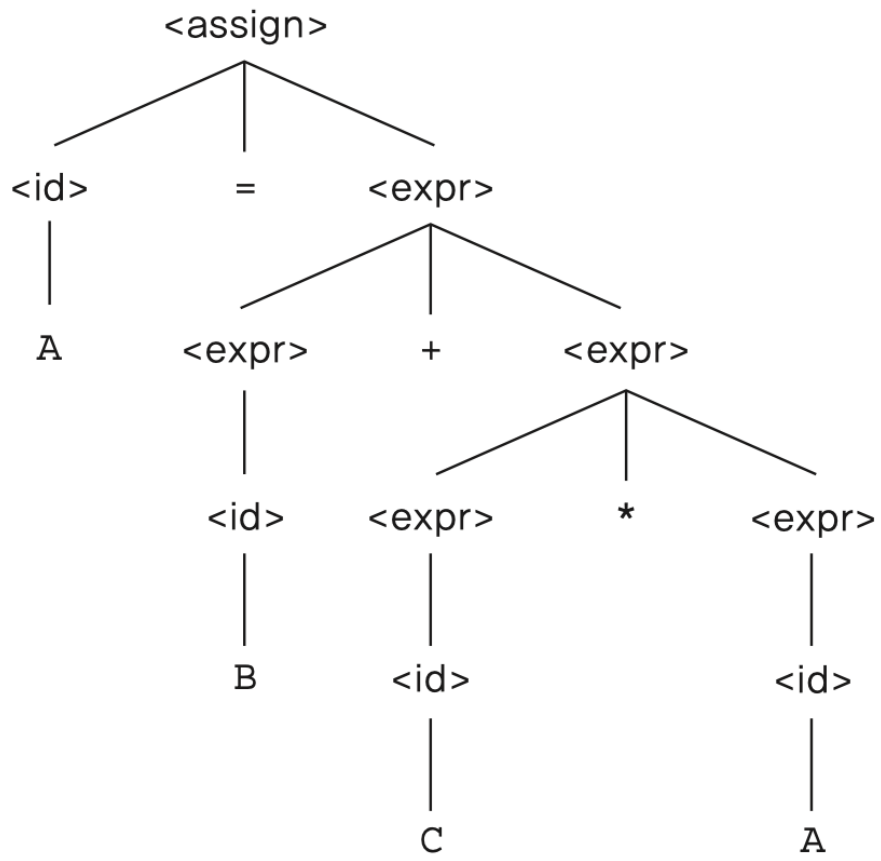
Ambiguity in Grammars

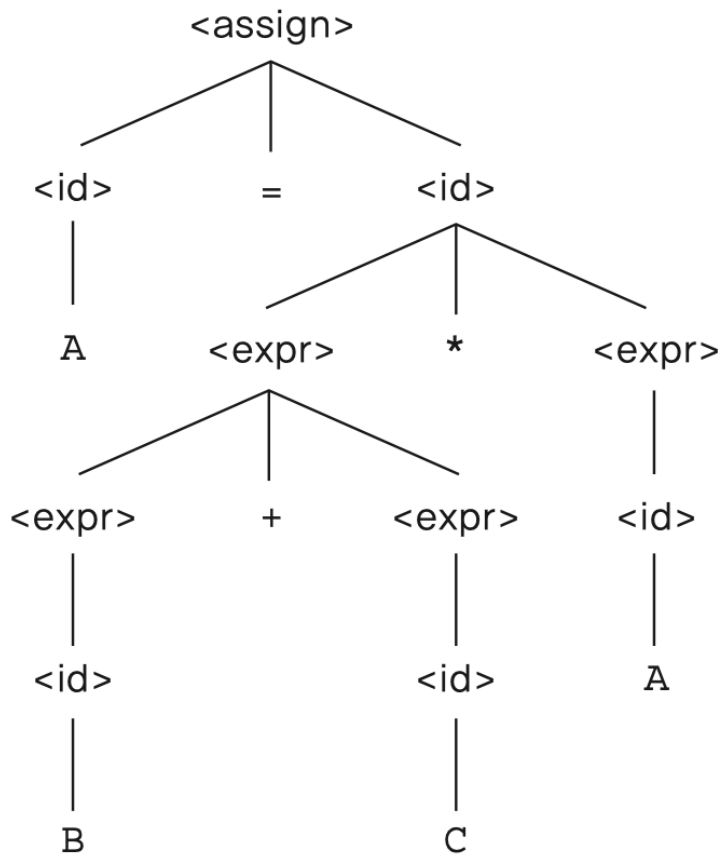
- **Example:**

```
<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <expr>
        | <expr> * <expr>
        | ( <expr> )
        | <id>
```

An Ambiguous Expression Grammar

- **Example:** `A = B + C * A`





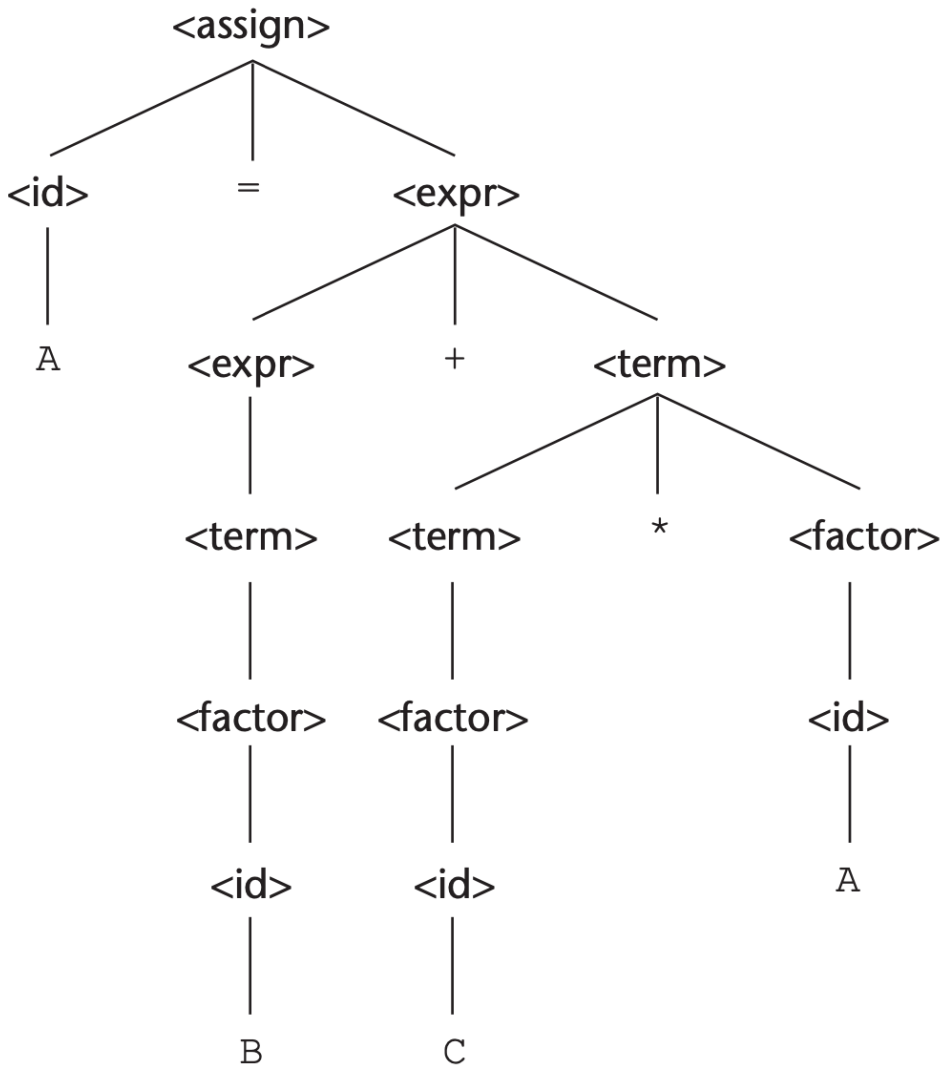
An Unambiguous Expression Grammar

- **If we use the parse tree to indicate precedence levels of the operators, there will be no ambiguity**
  - The correct ordering is specified by using separate nonterminal symbols
  - The higher precedence operator should be ***lower*** in the parse tree
- **Example:**

```
<assign> → <id> = <expr>
<id> → A | B | C
<expr> → <expr> + <term>
        | <term>
<term> → <term> * <factor>
        | <factor>
<factor> → ( <expr> )
        | <id>
```

An Unambiguous Expression Grammar

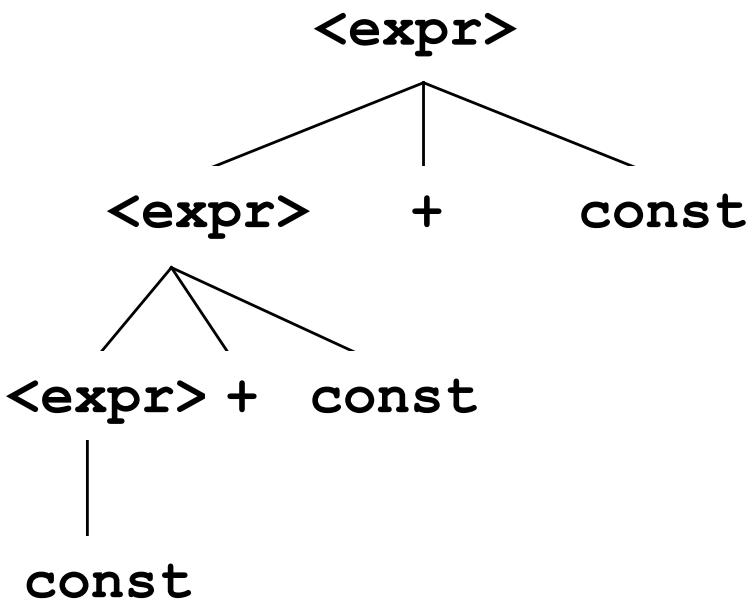
- **Example:** `A = B + C * A`



Associativity of Operators

- **Operator associativity can also be indicated by a grammar**
  - Example: `const + const + const`

```
<expr> → <expr> + <expr> | const   (ambiguous)
<expr> → <expr> + const  | const   (unambiguous)
```



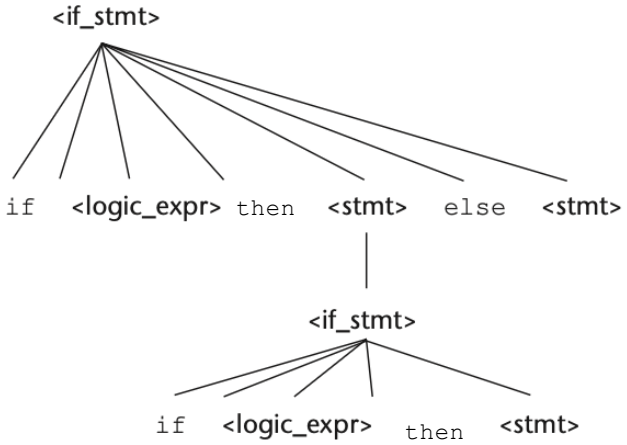
if-then-else (1/2)

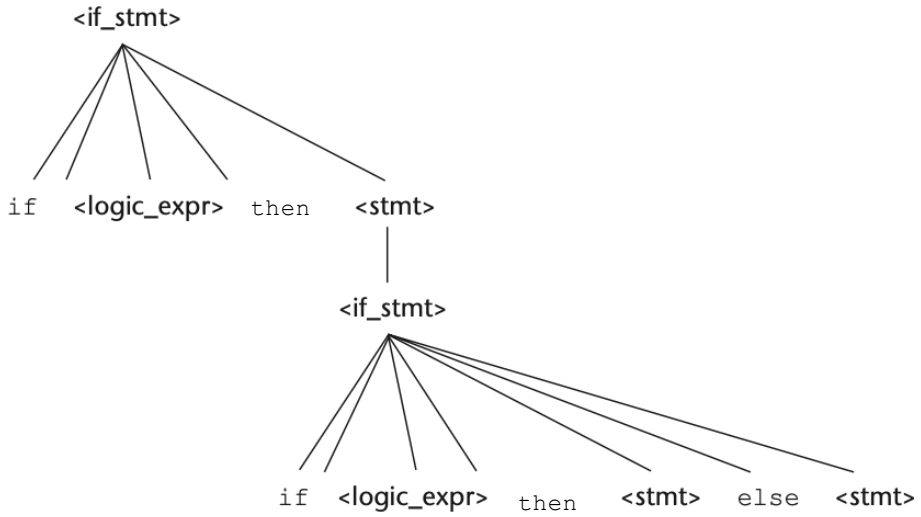
- **Java** `if-then-else` **grammar**

```
<if_stmt> → if (<logic_expr>) then <stmt>
          | if (<logic_expr>) then <stmt> else <stmt>
```

Ambiguous!

Example: `if <logic_expr> then if <logic_expr> then <stmt> else <stmt>`





if-then-else (2/2)

- **An unambiguous grammar for if-then-else**

```
<stmt>      → <matched>
            | <unmatched>

<matched>   → a non-if statement
            | if (<logic_expr>) then <matched> else <matched>

<unmatched> → if (<logic_expr>) then <stmt>
            | if (<logic_expr>) then <matched> else <unmatched>
```

Extended BNF (EBNF)

- **Optional parts are placed in brackets** `[ ]`

  `<if_stmt> → if (<expr>) <stmt> [else <stmt>]`

- **Repetitions (0 or more) are placed inside braces** `{ }`

  `<ident> → letter {letter|digit}`

- **Alternative parts of RHSs are placed inside parentheses and separated via vertical bars** `|`

  `<term> → <term> (+|-) const`

BNF and EBNF

- **BNF**

```
<expr> → <expr> + <term>
       | <expr> - <term>
       | <term>
<term> → <term> * <factor>
       | <term> / <factor>
       | <factor>
```

- **EBNF**

```
<expr> → <term> {(+ | -) <term>}
<term> → <factor> {(* | /) <factor>}
```

Recent Variations in EBNF

- **Alternative RHSs are put on separate lines**
  - Use of a colon :, instead of →
- **Use of `opt` for optional parts**
  - Constructor Declarator → SimpleName (FormalParameterList$_{\texttt{opt}}$)
- **Use of `one of` for choices**
  - AssignmentOperator → `one of` = *= /= %= ...

Static Semantics (정적 의미론)

- **Nothing to do with meaning**
- **Context-free grammars (CFGs) cannot describe all of the syntax of programming languages**
- **Categories of constructs that are trouble:**
  - Context-free, but cumbersome (e.g., types of operands in expressions)
  - Non-context-free (e.g., variables must be declared before they are used)
- **Static semantics**
  - Indirectly related to the meaning of programs (프로그램의 의미와 간접적으로 관련됨)
  - The legal forms of programs (프로그램의 적법한 형식)

Attribute Grammars (속성 문법)

- **Attribute grammars (AGs) have *additions* to CFGs to carry some semantic information on parse tree nodes**

- **Primary value of AGs:**
  - Static semantics specification (정적 의미론을 명세)
  - Compiler design (static semantics checking) (컴파일 시점에 검사)

Attribute Grammars: Definition (1/2)

- **Def.: An attribute grammar is a context-free grammar $G=(S,N,T,P)$ with the following *additions*:**
  - S: the start symbol (a single nonterminal from N)
  - N: the set of nonterminal symbols (variables)
  - T: the set of terminal symbols (tokens/lexemes)
  - P: the set of productions

  - Associated with each grammar symbol X is a set of attributes $A(X)$
  - Associated with each grammar rule is a set of semantic functions (의미 함수) and a possibly empty set of predicate functions (술어함수) over the attributes of the symbols in the grammar rule.
  - A predicate function has the form of a Boolean expression on the union of the attribute set $\{A(X_0), \ldots, A(X_n)\}$ and a set of literal attribute values
  - The parse tree is fully attributed if all the attribute values are computed

Attribute Grammars: Definition (2/2)

- Let $X_0 \rightarrow X_1 \ \ldots \ X_n$ be a rule
- Functions of the form $S(X_0) = f(A(X_1), \ldots, A(X_n))$ define synthesized (합성) attributes
- Functions of the form $I(X_j) = f(A(X_0), \ldots, A(X_n))$, for $1 \leq j \leq n$, define inherited (상속) attributes
- Initially, there are intrinsic (내장) attributes on the leaves
- A predicate function has the form of a Boolean expression on the union of the attribute set $\{A(X_0), \ldots, A(X_n)\}$ and a set of literal attribute values

Attribute Grammars: An Example (1/2)

- **Syntax**

```
<assign> → <var> = <expr>
<expr>   → <var> + <var>
         | <var>
<var>    → A | B | C
```

- `actual_type`**: synthesized for** `<var>` **and** `<expr>`
- `expected_type`**: inherited for** `<expr>`

Attribute Grammars: An Example (2/2)

- **Syntax rule**

  `<expr> → <var>[1] + <var>[2]`

  - Semantic rules:

    `<expr>.actual_type ← <var>[1].actual_type`

  - Predicate:

    `<var>[1].actual_type == <var>[2].actual_type`

    `<expr>.expected_type == <expr>.actual_type`

- **Syntax rule**

  `<var> → A | B | C`

  - Semantic rule:

    `<var>.actual_type ← lookup (<var>.string)`

    - The look-up function looks up a given variable name in the symbol table and returns the variable's type

Attribute Grammars

- **How are attribute values computed?**
  - If all attributes were inherited, the tree could be decorated in top-down order
  - If all attributes were synthesized, the tree could be decorated in bottom-up order
  - In many cases, both kinds of attributes are used, and it is some combination of top-down and bottom-up that must be used

Attribute Grammars

<expr>.expected_type ← inherited from parent

<expr>.expected_type = <var>.actual_type

<var>[1].actual_type ← lookup (A)

<var>[2].actual_type ← lookup (B)

<var>[1].actual_type =? <var>[2].actual_type

<expr>.actual_type ← <var>[1].actual_type

<expr>.actual_type =? <expr>.expected_type

Q & A